

Conception et Mise en œuvre d'une Infrastructure Virtualisée Sécurisée Orientée Big Data

Table des matières

1	Introduction	3
1.1	Contexte du Projet	3
1.2	Objectifs	3
1.3	Environnement Matériel et Logiciel	3
2	Architecture de l'Infrastructure	4
2.1	Vue d'Ensemble	4
2.2	Schéma de l'Architecture	5
2.3	Configuration Réseau	5
3	Virtualisation Réseau et Segmentation	7
3.1	Concept	7
3.2	Segmentation par Pare-feu UFW	7
3.2.1	vm-appserver	8
3.2.2	vm-dbase	9
3.2.3	vm-proces	9
3.2.4	vm-bastion	10
4	Virtualisation du Stockage	11
4.1	Snapshots VirtualBox	11
4.2	Stockage Partagé NFS (Simulation NAS)	11
4.2.1	Concept	11
4.2.2	Configuration du Serveur NFS (vm-dbase)	12
4.2.3	Montage sur les Clients (vm-appserver et vm-proces)	12
4.2.4	Démonstration	12
5	Conteneurisation avec Docker	13
5.1	Concept	13
5.2	Installation de Docker sur vm-appserver	13
5.3	Structure de l'Application	13
5.3.1	Dockerfile	13
5.3.2	docker-compose.yml	13
5.4	Construction et Démarrage	14
6	Sécurisation de l'Infrastructure	15
6.1	Bastion Host (vm-bastion)	15
6.2	Authentification par Clés SSH	15
6.3	HTTPS avec Nginx et TLS	15
6.4	Contrôle d'Accès PostgreSQL	17

7	Application Big Data	18
7.1	Description du Cas d'Usage	18
7.2	Flux de Données	18
7.3	API Flask (vm-appserver)	19
7.4	Base de Données PostgreSQL (vm-dbase)	20
7.5	Service Analytique Python/Pandas (vm-proces)	20
7.5.1	Statistiques Descriptives (/stats)	21
7.5.2	Détection d'Anomalies (/anomalies)	21
7.6	Tableau de Bord Web	21
8	Monitoring avec Netdata	23
8.1	Concept	23
8.2	Architecture de Monitoring	23
8.3	Résultats	23
9	Résultats et Validation	25
9.1	Tests de Validation	25
9.2	Couverture des Exigences	26
10	Conclusion	27

1. Introduction

1.1 Contexte du Projet

Dans le cadre du module de virtualisation, nous avons conçu et mis en œuvre une infrastructure virtualisée complète sur une machine personnelle (ASUS TUF A15, Windows 11, 16 Go de RAM), en utilisant VirtualBox comme hyperviseur de type 2.

L'objectif principal est de simuler une infrastructure d'entreprise réelle comprenant plusieurs serveurs virtuels interconnectés, sécurisés et spécialisés, avec une application de traitement de données de type Big Data comme cas d'usage concret.

1.2 Objectifs

béployer et configurer plusieurs machines virtuelles avec des rôles distincts

Implémenter la virtualisation réseau via la segmentation et l'isolement

Mettre en place la virtualisation du stockage via NFS et les snapshots

Conteneuriser une application web avec Docker

Sécuriser l'infrastructure : pare-feu, SSH, HTTPS, bastion host

Surveiller l'infrastructure en temps réel via Netdata

Démontrer un cas d'usage Big Data : ingestion, stockage et analyse de données

1.3 Environnement Matériel et Logiciel

Machine hôte	ASUS TUF A15 — Windows 11
RAM totale	16 Go (8 Go alloués aux VMs, 8 Go pour Windows)
Hyperviseur	Oracle VirtualBox
OS invité	Ubuntu Server 22.04 LTS (sans interface graphique)
Conteneurisation	Docker + Docker Compose
Base de données	PostgreSQL 14
Serveur web	Nginx (reverse proxy) + Flask (API)
Monitoring	Netdata

TABLE 1.1 – Environnement technique du projet

2. Architecture de l'Infrastructure

2.1 Vue d'Ensemble

L'infrastructure est composée de **4 machines virtuelles** communiquant via un réseau interne virtuel nommé `bigdata-net`. Chaque VM possède un rôle précis et unique.

vm-bastion	1.5 Go	1	12 Go	192.168.50.5	Bastion host + Monitoring Net-data
vm-appserver	3 Go	2	25 Go	192.168.50.10	Docker + Flask API + Nginx HTTPS
vm-dbase	2 Go	1	25 Go	192.168.50.20	PostgreSQL + Serveur NFS
vm-proces	3 Go	2	25 Go	192.168.50.30	Analytique Python/Pandas

TABLE 2.1 – Configuration des machines virtuelles

2.2 Schéma de l'Architecture

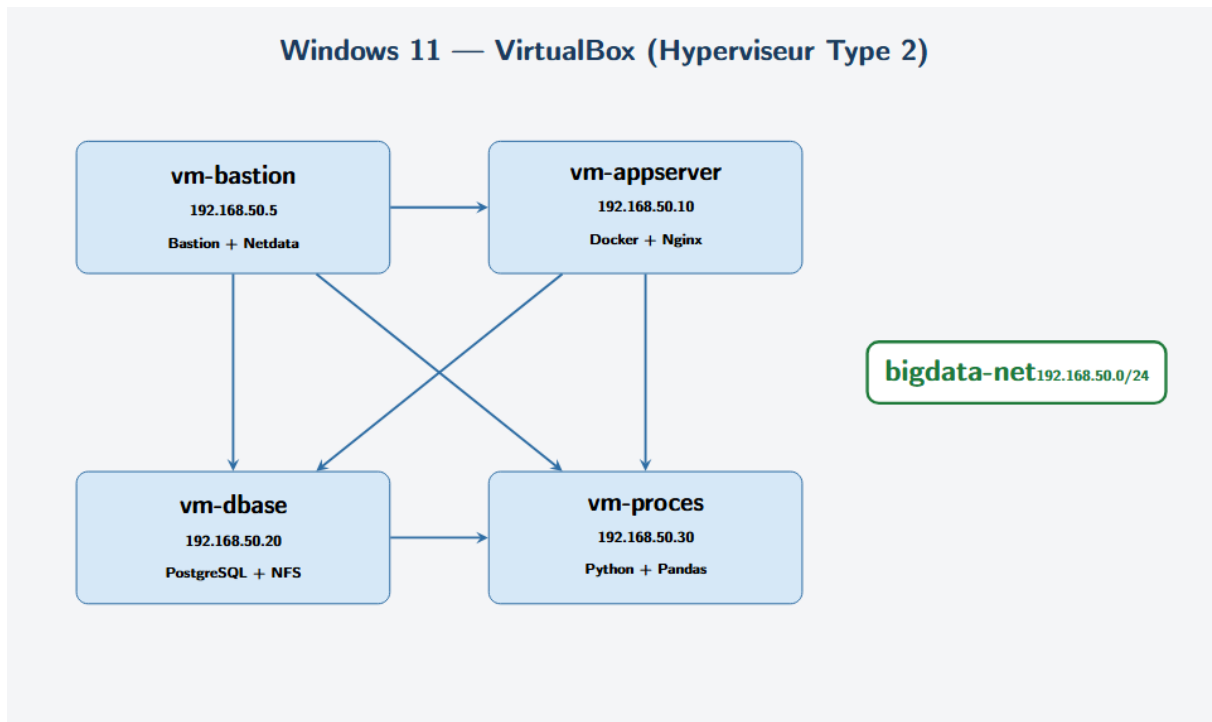


FIGURE 2.1 – Schéma de l'architecture réseau et des machines virtuelles

2.3 Configuration Réseau

Chaque VM dispose de **deux adaptateurs réseau** :

- **Adaptateur 1 — NAT** : fournit un accès internet à la VM (pour `apt install`, `pip`, etc.)
- **Adaptateur 2 — Internal Network (**bigdata-net**)** : réseau privé pour la communication entre VMs, complètement isolé de l'extérieur

Les adresses IP statiques sont configurées via **Netplan** sur chaque VM, en désactivant `cloud-init` pour éviter l'écrasement des configurations au redémarrage.

```
ossama@vm-db:/etc/netplan$ sudo nano 50-cloud-init.yaml
[sudo] password for ossama:
ossama@vm-db:/etc/netplan$ sudo netplan apply
WARNING:root:Cannot call Open vSwitch: ovssdb-server.service is not running.
ossama@vm-db:/etc/netplan$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group de
fault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state U
P group default qlen 1000
    link/ether 08:00:27:b4:f3:13 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 86397sec preferred_lft 86397sec
    inet6 fd17:625c:f037:2:a00:27ff:feb4:f313/64 scope global dynamic mngtmp
addr noprefixroute
        valid_lft 86398sec preferred_lft 14398sec
    inet6 fe80::a00:27ff:feb4:f313/64 scope link
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state U
P group default qlen 1000
    link/ether 08:00:27:cd:ce:eb brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.20/24 brd 192.168.50.255 scope global enp0s8
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fecd:ceeb/64 scope link
        valid_lft forever preferred_lft forever
```

FIGURE 2.2 – exemple d'adresse ip des interface de vm database

3. Virtualisation Réseau et Segmentation

3.1 Concept

La virtualisation réseau consiste à créer des réseaux logiques indépendants de l'infrastructure physique. Dans notre projet, le réseau **bigdata-net** est un réseau entièrement virtuel qui n'existe qu'à l'intérieur de VirtualBox — aucun câble physique, aucun switch physique.

3.2 Segmentation par Pare-feu UFW

Chaque VM possède des règles UFW (Uncomplicated Firewall) strictes qui définissent précisément quels services sont accessibles et depuis quelles adresses IP.

3.2.1 vm-appserver

```
ossama@vm-appserver:~$ sudo ufw default deny incoming
[sudo] password for ossama:
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
ossama@vm-appserver:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
ossama@vm-appserver:~$ sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw allow 5000/tcp
sudo ufw enable
sudo ufw status verbose
Rules updated
Rules updated (v6)
Rules updated
Rules updated (v6)
Rules updated
Rules updated (v6)
Rules updated
Rules updated (v6)
Command may disrupt existing ssh connections. Proceed with operation (y|n)?
y
Firewall is active and enabled on system startup
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
80/tcp ALLOW IN Anywhere
443/tcp ALLOW IN Anywhere
5000/tcp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)
80/tcp (v6) ALLOW IN Anywhere (v6)
443/tcp (v6) ALLOW IN Anywhere (v6)
5000/tcp (v6) ALLOW IN Anywhere (v6)

ossama@vm-appserver:~$
```

FIGURE 3.1 – les regles de firewall dans vm appserver

3.2.2 vm-dbase

```
ossama@vm-db:~$ sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow from 192.168.50.10 to any port 5432 # only app server can r
each DB
sudo ufw enable
[sudo] password for ossama:
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
Rules updated
Rules updated (v6)
Rules updated
Command may disrupt existing ssh connections. Proceed with operation (y|n)?
y
Firewall is active and enabled on system startup
ossama@vm-db:~$
```

FIGURE 3.2 – les regles de firewall dans vm db

3.2.3 vm-proces

```
ossama@vm-processing:~$ sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow from 192.168.50.10 to any port 8000
sudo ufw enable
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
Rules updated
Rules updated (v6)
Rules updated
Command may disrupt existing ssh connections. Proceed with operation (y|n)?
y
Firewall is active and enabled on system startup
ossama@vm-processing:~$ |
```

FIGURE 3.3 – les regles de firewall dans vm db

3.2.4 vm-bastion

```
connection to 192.168.50.30 closed.  
ossama@bastion:~$ sudo ufw default deny incoming  
sudo ufw default allow outgoing  
sudo ufw allow 22/tcp  
sudo ufw enable  
sudo ufw status  
[sudo] password for ossama:  
Default incoming policy changed to 'deny'  
(be sure to update your rules accordingly)  
Default outgoing policy changed to 'allow'  
(be sure to update your rules accordingly)  
Rules updated  
Rules updated (v6)  
Command may disrupt existing ssh connections. Proceed with operation (y|n)?  
y  
Firewall is active and enabled on system startup  
Status: active  
  
To Action From  
--  
22/tcp ALLOW Anywhere  
22/tcp (v6) ALLOW Anywhere (v6)  
  
ossama@bastion:~$
```

FIGURE 3.4 – les regles de firewall dans vm bastion

Résultat : PostgreSQL (port 5432) n'est accessible que depuis 192.168.50.10 et 192.168.50.30. L'API analytique (port 8000) n'est accessible que depuis 192.168.50.10. Cette segmentation simule un vrai réseau d'entreprise segmenté par VLANs.

4. Virtualisation du Stockage

4.1 Snapshots VirtualBox

Les snapshots permettent de sauvegarder l'état complet d'une VM (disque, RAM, processus) à un instant donné. Ils constituent le mécanisme de sauvegarde et de reprise sur sinistre de notre infrastructure.

Nous avons pris des snapshots à chaque étape clé du projet :

VM	Nom du Snapshot	Date
vm-appserver	bastion added	26/04/2026
vm-appserver	finale + dashboard	26/04/2026
vm-dbase	Clean PostgreSQL Install	25/04/2026
vm-proces	Final : Analytics service running	26/04/2026

TABLE 4.1 – Snapshots pris durant le projet

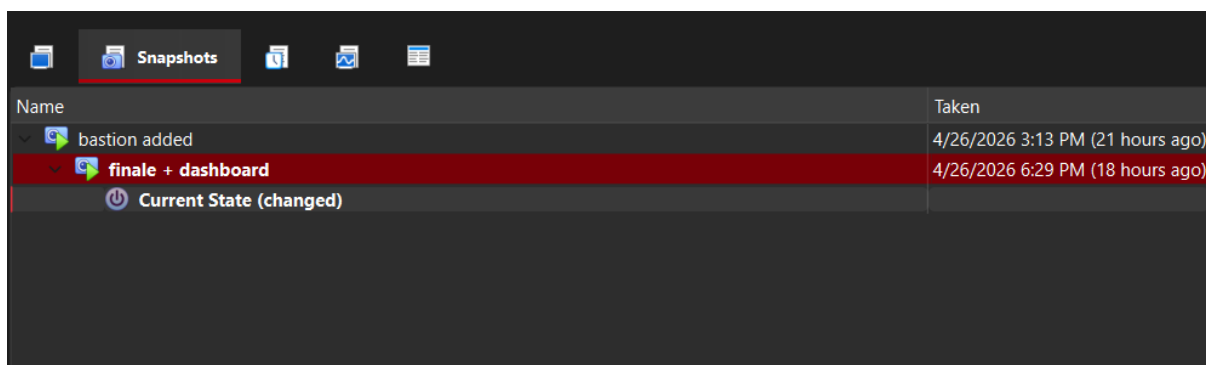


FIGURE 4.1 – Capture d'écran de l'interface de gestion des snapshots

4.2 Stockage Partagé NFS (Simulation NAS)

4.2.1 Concept

NFS (Network File System) permet à plusieurs machines de partager un répertoire via le réseau. `vm-dbase` joue le rôle de **serveur NAS**, et `vm-appserver` ainsi que `vm-proces` montent ce répertoire partagé.

4.2.2 Configuration du Serveur NFS (vm-dbase)

```
sudo apt install nfs-kernel-server -y
sudo mkdir -p /srv/bigdata-share
sudo chown nobody:nogroup /srv/bigdata-share
sudo chmod 777 /srv/bigdata-share

# /etc/exports :
/srv/bigdata-share 192.168.50.10(rw,sync,no_subtree_check)
/srv/bigdata-share 192.168.50.30(rw,sync,no_subtree_check)

sudo exportfs -a
sudo systemctl restart nfs-kernel-server
```

4.2.3 Montage sur les Clients (vm-appserver et vm-proces)

```
ossama@vm-appserver:~$ sudo mkdir -p /mnt/bigdata-share
ossama@vm-appserver:~$ sudo mount 192.168.50.20:/srv/bigdata-share /mnt/bigdata-share
ossama@vm-appserver:~$ df -h | grep bigdata
192.168.50.20:/srv/bigdata-share 25G 3.7G 20G 16% /mnt/bigdata-share
ossama@vm-appserver:~$
```

4.2.4 Démonstration

vm-proces génère un rapport analytique et l'écrit dans le stockage partagé, que vm-appserver peut lire instantanément :

```
ossama@vm-processing:~$ sudo systemctl restart analytics
ossama@vm-processing:~$ curl http://192.168.50.30:8000/report
{"path":"/mnt/bigdata-share/report.txt","status":"report saved"}
ossama@vm-processing:~$ cat /mnt/bigdata-share/report.txt
=== BigData Analytics Report ===
Generated at: 2026-04-25 18:15:37.071670
Total records: 200

      count    mean    min    max
sensor_id
humidity_01    43  30.99  18.12  146.60
pressure_01    45  26.89  18.21  141.36
temp_001       36  29.94  18.10  140.52
temp_002       32  24.54  18.41   29.34
vibration_01   44  26.17  18.03  122.12
ossama@vm-processing:~$
```

Résultat : Le fichier écrit par vm-proces est immédiatement lisible par vm-appserver via le montage NFS — démonstration concrète du stockage partagé de type NAS.

5. Conteneurisation avec Docker

5.1 Concept

Docker permet d'encapsuler une application et toutes ses dépendances dans un conteneur isolé et portable. Contrairement à une VM qui virtualise un ordinateur entier, un conteneur ne virtualise que la couche applicative — beaucoup plus léger et rapide.

5.2 Installation de Docker sur vm-appserver

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker ossama
sudo apt install docker-compose -y
```

5.3 Structure de l'Application

L'application Flask est conteneurisée avec les fichiers suivants :

5.3.1 Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
```

5.3.2 docker-compose.yml

```
version: "3.8"
services:
  web:
    build: .
    ports:
```

```

- "5000:5000"
environment:
- DB_HOST=192.168.50.20
- DB_NAME=bigdata
- DB_USER=bigdata_user
- DB_PASS=SecurePass123
restart: unless-stopped

```

5.4 Construction et Démarrage

```

ossama@vm-appserver:~/bigdata-app$ docker compose up -d --build
WARN[0000] /home/ossama/bigdata-app/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Building 27.0s (12/12) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 496B 0.0s
=> [internal] load build definition from Dockerfile 0.1s
=> => transferring dockerfile: 229B 0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 2.0s
=> [internal] load .dockerignore 0.1s
=> => transferring context: 2B 0.0s
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:6d85378d88a 11.2s
=> => resolve docker.io/library/python:3.11-slim@sha256:6d85378d88a1 0.1s
=> => sha256:7ccd73948dde4b7f1623ac6bf9b58706fb83d0a363d 249B / 249B 0.3s
=> => sha256:f3ba2250c52436e41e9f2f3830af92de466fc 14.37MB / 14.37MB 3.4s
=> => sha256:91ff8760033c3b9bc9c3a3bcc6ff66c113ec9e1 1.29MB / 1.29MB 1.2s
=> => sha256:3531af2bc2a9c8883754652783cf96207d531 29.78MB / 29.78MB 6.1s
=> => extracting sha256:3531af2bc2a9c8883754652783cf96207d53189db279 2.1s
=> => extracting sha256:91ff8760033c3b9bc9c3a3bcc6ff66c113ec9e13f94f 0.3s
=> => extracting sha256:f3ba2250c52436e41e9f2f3830af92de466fcacc2d84 2.1s
=> => extracting sha256:7ccd73948dde4b7f1623ac6bf9b58706fb83d0a363d7 0.1s
=> [internal] load build context 0.2s
=> => transferring context: 1.39kB 0.0s
=> [2/5] WORKDIR /app 0.5s
=> [3/5] COPY requirements.txt . 0.1s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt 8.1s
=> [5/5] COPY app.py . 0.2s
=> exporting to image 3.8s
=> => exporting layers 2.2s
=> => exporting manifest sha256:9c31e95bf21ceb247991ba0dfdb4b0dee2dd 0.0s
=> => exporting config sha256:5ccad71b14fcfd531660f108196bccdb06040 0.0s
=> => exporting attestation manifest sha256:8e00f0798424a7ddb15976bd 0.1s
=> => exporting manifest list sha256:75117640cc51de39fc0ba4f2abd772f 0.0s
=> => naming to docker.io/library/bigdata-app-web:latest 0.0s
=> => unpacking to docker.io/library/bigdata-app-web:latest 1.3s

```

6. Sécurisation de l'Infrastructure

6.1 Bastion Host (vm-bastion)

Le bastion host est le **point d'entrée unique** de l'infrastructure. Depuis Windows, seul le bastion est accessible directement via SSH. Les autres VMs ne sont joignables qu'en passant par lui.

```
# Architecture d'accès :
Windows (PowerShell)
|
| ssh -p 2225 ossama@127.0.0.1
v
vm-bastion (192.168.50.5)
|-- ssh ossama@192.168.50.10 --> vm-appserver
|-- ssh ossama@192.168.50.20 --> vm-dbase
|-- ssh ossama@192.168.50.30 --> vm-proces
```

6.2 Authentification par Clés SSH

L'authentification par mot de passe est désactivée sur toutes les VMs. Seule l'authentification par clé SSH est autorisée.

```
# Example of overriding settings on a per-user basis
#Match User anoncvs
#    X11Forwarding no
#    AllowTcpForwarding no
#    PermitTTY no
#    ForceCommand cvs server
PasswordAuthentication no
PermitRootLogin no
MaxAuthTries 3|
```

6.3 HTTPS avec Nginx et TLS

Nginx est installé sur **vm-appserver** comme reverse proxy et gère le chiffrement TLS. Un certificat auto-signé valable 365 jours a été généré avec OpenSSL.

[illegible]

La configuration Nginx redirige automatiquement HTTP vers HTTPS et n'autorise que les protocoles TLS 1.2 et 1.3 :

```
GNU nano 6.2 /etc/nginx/sites-available/bigdata-app *
server {
    listen 80;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    ssl_certificate /etc/nginx/ssl/certificate.crt;
    ssl_certificate_key /etc/nginx/ssl/private.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    add_header Strict-Transport-Security "max-age=31536000";
    add_header X-Frame-Options "SAMEORIGIN";

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

6.4 Contrôle d'Accès PostgreSQL

En plus du pare-feu UFW, PostgreSQL lui-même contrôle les accès via `pg_hba.conf` :

```
# /etc/postgresql/14/main/pg_hba.conf :
host bigdata bigdata_user 192.168.50.10/32 scram-sha-256
host bigdata bigdata_user 192.168.50.30/32 scram-sha-256
```

Couche de Sécurité	Implémentation
Accès SSH	Clés ed25519, mot de passe désactivé
Pare-feu réseau	UFW sur chaque VM avec règles strictes par IP
Chiffrement web	TLS 1.2/1.3 via Nginx + certificat OpenSSL
Accès base de données	pg_hba.conf + UFW — IP spécifiques uniquement
Point d'entrée unique	Bastion host — aucun accès direct aux VMs internes

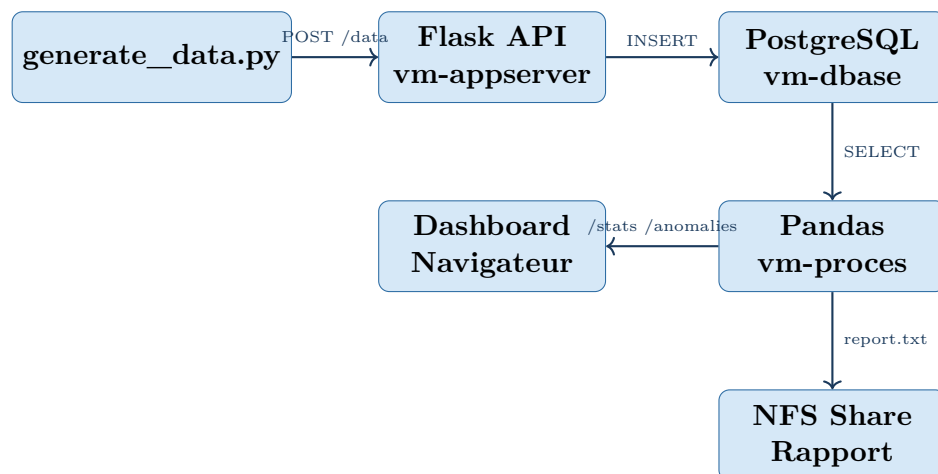
TABLE 6.1 – Récapitulatif des couches de sécurité

7. Application Big Data

7.1 Description du Cas d'Usage

L'application simule un pipeline IoT de collecte et analyse de données de capteurs. Cinq capteurs (`temp_001`, `temp_002`, `humidity_01`, `pressure_01`, `vibration_01`) génèrent des mesures qui transitent dans le pipeline complet.

7.2 Flux de Données



```

ossama@vm-appserver:~/bigdata-app$ curl http://192.168.50.30:8000/stats | py
thon3 -m json.tool
curl http://192.168.50.30:8000/anomalies | python3 -m json.tool
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Curr
ent
                                Dload  Upload  Total  Spent  Left  Spee
d
  0      0    0     0    0     0     0      0  --:--:-- --:--:-- --:--:--
100  589  100  589    0     0  7863      0  --:--:-- --:--:-- --:--:-- 79
59
{
  "generated_at": "2026-04-25T15:57:01.544018",
  "sensors": [
    {
      "count": 43,
      "max": 146.6,
      "mean": 30.99,
      "median": 23.46,
      "min": 18.12,
      "sensor_id": "humidity_01",
      "std": 28.67
    },
    {
      "count": 45,
      "max": 141.36,
      "mean": 26.89,
      "median": 23.61,
      "min": 18.21,
      "sensor_id": "pressure_01",
      "std": 17.79
    }
  ]
}

```

7.3 API Flask (vm-appserver)

Endpoint	Méthode	Description
/health	GET	Vérification de l'état du service
/data	POST	Ingestion d'une mesure capteur
/data	GET	Récupération des 100 dernières mesures
/stats	GET	Proxy vers les statistiques (vm-proces)
/anomalies	GET	Proxy vers la détection d'anomalies
/dashboard	GET	Tableau de bord web temps réel

TABLE 7.1 – Endpoints de l'API Flask

```

ossama@vm-appserver:~/bigdata-app$ sudo chmod 600 /etc/nginx/ssl/private.key
ossama@vm-appserver:~/bigdata-app$ sudo nano /etc/nginx/sites-available/bigdata-app
ossama@vm-appserver:~/bigdata-app$ sudo ln -s /etc/nginx/sites-available/bigdata-app /etc/nginx/sites-enabled/
ossama@vm-appserver:~/bigdata-app$ sudo rm /etc/nginx/sites-enabled/default
ossama@vm-appserver:~/bigdata-app$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
ossama@vm-appserver:~/bigdata-app$ sudo systemctl restart nginx
ossama@vm-appserver:~/bigdata-app$ curl -k https://192.168.50.10/health
{"service":"app-server","status":"ok"}
ossama@vm-appserver:~/bigdata-app$

```

7.4 Base de Données PostgreSQL (vm-dbase)

```

CREATE TABLE sensor_data (
    id          SERIAL PRIMARY KEY,
    sensor_id   VARCHAR(50) NOT NULL,
    value       DECIMAL(10, 2) NOT NULL,
    timestamp   TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

```

200 enregistrements ont été insérés via le script `generate_data.py` pour la démonstration.

7.5 Service Analytique Python/Pandas (vm-proces)

Le service analytique expose deux fonctionnalités principales :

```

ics.py
(analytics-env) ossama@vm-processing:~$ sudo systemctl daemon-reload
(analytics-env) ossama@vm-processing:~$ sudo systemctl restart analytics
(analytics-env) ossama@vm-processing:~$ sudo systemctl status analytics
● analytics.service - Big Data Analytics Service
   Loaded: loaded (/etc/systemd/system/analytics.service; enabled; vendor>
   Active: active (running) since Sat 2026-04-25 15:23:36 UTC; 5s ago
     Main PID: 17542 (python)
        Tasks: 2 (limit: 3422)
      Memory: 46.3M
         CPU: 1.167s
    CGroup: /system.slice/analytics.service
            └─17542 /home/ossama/analytics-env/bin/python analytics.py

Apr 25 15:23:36 vm-processing systemd[1]: Started Big Data Analytics Servic>
Apr 25 15:23:37 vm-processing python[17542]: * Serving Flask app 'analytic>
Apr 25 15:23:37 vm-processing python[17542]: * Debug mode: off
Apr 25 15:23:37 vm-processing python[17542]: WARNING: This is a development>
Apr 25 15:23:37 vm-processing python[17542]: * Running on all addresses (0>
Apr 25 15:23:37 vm-processing python[17542]: * Running on http://127.0.0.1>
Apr 25 15:23:37 vm-processing python[17542]: * Running on http://10.0.2.15>
Apr 25 15:23:37 vm-processing python[17542]: Press CTRL+C to quit
lines 1-18/18 (END)

```

7.5.1 Statistiques Descriptives (/stats)

Calcul de statistiques par capteur sur les dernières 72 heures : count, mean, median, std, min, max.

```
# Resultat :
{"sensors": [
  {"sensor_id": "humidity_01", "count": 43, "mean": 30.99,
   "median": 23.46, "std": 28.67, "min": 18.12, "max": 146.6},
  {"sensor_id": "pressure_01", "count": 45, "mean": 26.89,
   ...},
  {"sensor_id": "temp_001", "count": 36, "mean": 29.94,
   ...},
  {"sensor_id": "temp_002", "count": 32, "mean": 24.54,
   ...},
  {"sensor_id": "vibration_01", "count": 44, "mean": 26.17,
   ...}
], "total_records": 200}
```

7.5.2 Détection d'Anomalies (/anomalies)

Utilisation de la méthode du **Z-score** : une valeur est considérée anormale si elle s'écarte de plus de 2 écarts-types de la moyenne.

```
# Anomalies detectees :
{"anomalies": [
  {"sensor_id": "humidity_01", "value": 146.6, "z_score":
   4.03},
  {"sensor_id": "humidity_01", "value": 143.83, "z_score":
   3.94},
  {"sensor_id": "pressure_01", "value": 141.36, "z_score":
   6.43},
  {"sensor_id": "temp_001", "value": 140.52, "z_score":
   4.05},
  {"sensor_id": "vibration_01", "value": 122.12, "z_score":
   6.30}
], "count": 7}
```

7.6 Tableau de Bord Web

Un dashboard HTML/JavaScript auto-rafraîchi toutes les 10 secondes affiche :

- Les statistiques par capteur sous forme de tableau
- Les anomalies détectées surlignées en rouge
- Les 20 dernières mesures en temps réel

Accessible depuis Windows via : <https://localhost:8443/dashboard>



8. Monitoring avec Netdata

8.1 Concept

Netdata est un outil de monitoring open-source qui collecte des métriques système en temps réel (CPU, RAM, disque, réseau, processus) et les affiche dans un tableau de bord interactif.

8.2 Architecture de Monitoring

Chaque VM exécute un **agent Netdata** qui envoie ses métriques au bastion via un mécanisme de streaming. Le bastion agrège et affiche tout dans une interface unique.

```
# Configuration streaming sur vm-appserver, vm-dbase, vm-proces
:
# /etc/netdata/stream.conf :
[stream]
    enabled = yes
    destination = 192.168.50.5:19999
    api key = bigdata-monitoring-key

# Configuration reception sur vm-bastion :
[bigdata-monitoring-key]
    enabled = yes
    allow from = 192.168.50.*
```

8.3 Résultats

Le dashboard Netdata sur le bastion affiche **4 nœuds en temps réel** : bastion, vm-appserver, vm-dbase, vm-proces — avec pour chacun : CPU, RAM utilisée, lectures/écritures disque, trafic réseau entrant/sortant.

Accessible depuis Windows via : <http://localhost:19999>



LIVE WEBINAR | Netdata Live Functions for DB & Network Monitoring — save your seat for April 29 →

local | Playing | 4/26/26 • 18:04

Servers

Total Servers: 4 | Live: 4 | Stale: 0 | Off: 0

Search or filter...

Server	Alerts	Address	Uptime	CPU	Memory	Load	Disk Read	Disk Write	Acti
Live (4)									
bastion		10.0.2.15	44:48.72	24%	0.5/1.7 GiB	28%	0.6	33.9 KiB/s	372.0 KiB/s
vm-appserver		10.0.2.15	45:40.96	17%	0.5/2.9 GiB	18%	1.2	0.6 KiB/s	34.9 KiB/s
vm-db		10.0.2.15	52:38.08	7%	0.4/2.9 GiB	14%	0.3	0.0 KiB/s	11.5 KiB/s
vm-processing		10.0.2.15	51:18.21	7%	0.5/2.3 GiB	20%	0.3	0.0 KiB/s	12.2 KiB/s

9. Résultats et Validation

9.1 Tests de Validation

```
ossama@vm-appserver:~$ cat /mnt/bigdata-share/report.txt
=== BigData Analytics Report ===
Generated at: 2026-04-25 18:15:37.071670
Total records: 200

count    mean    min    max
sensor_id
humidity_01    43    30.99    18.12    146.60
pressure_01    45    26.89    18.21    141.36
temp_001       36    29.94    18.10    140.52
temp_002       32    24.54    18.41    29.34
vibration_01   44    26.17    18.03    122.12
ossama@vm-appserver:~$ |
```

```
ossama@vm-appserver:~$ docker compose -f ~/bigdata-app/docker-compose.yml ps
WARN[0000] /home/ossama/bigdata-app/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please
remove it to avoid potential confusion
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
bigdata-app-web-1   bigdata-app-web     "gunicorn --bind 0.0...  web        35 minutes ago Up 35 minutes 0.0.0.0:5000->
5000/tcp, [::]:5000->5000/tcp
ossama@vm-appserver:~$ curl http://localhost:5000/health
{"service": "app-server", "status": "ok"}
ossama@vm-appserver:~$ curl -k https://192.168.50.10/health
{"service": "app-server", "status": "ok"}
ossama@vm-appserver:~$ curl -X POST http://localhost:5000/data \
-H "Content-Type: application/json" \
-d '{"sensor_id": "test_demo", "value": 25.5}'
{"status": "inserted"}
ossama@vm-appserver:~$ curl http://192.168.50.30:8000/stats | python3 -m json.tool
{
  "generated_at": "2026-04-28T21:05:27.661730",
  "sensors": [
    {
      "count": 30,
      "max": 138.5,
      "mean": 28.36,
      "median": 24.72,
      "min": 19.16,
      "sensor_id": "humidity_01",
```

```

ossama@vm-db:~$ sudo -u postgres psql -d bigdata -c "SELECT sensor_id, value, timestamp FROM sensor_data ORDER BY timestamp DESC LIMIT 5;"
could not change directory to "/home/ossama": Permission denied
  sensor_id | value | timestamp
-----+-----+-----
temp_002   | 29.27 | 2026-04-28 20:44:48.24967+00
pressure_01 | 18.42 | 2026-04-28 20:44:48.145993+00
temp_002   | 20.86 | 2026-04-28 20:44:48.041134+00
temp_002   | 23.27 | 2026-04-28 20:44:47.932312+00
vibration_01 | 27.38 | 2026-04-28 20:44:47.831449+00
(5 rows)

ossama@vm-db:~$ sudo -u postgres psql -d bigdata -c "SELECT COUNT(*) FROM sensor_data;"
[sudo] password for ossama:
could not change directory to "/home/ossama": Permission denied
  count
-----
    401
(1 row)

ossama@vm-db:~$ |

```

9.2 Couverture des Exigences

Exigence	Implémentation
Virtualisation (VMs)	4 VMs sur VirtualBox
Virtualisation réseau	bigdata-net, segmentation UFW
Virtualisation stockage	NFS partagé + snapshots
Conteneurisation	Docker + Compose sur vm-appserver
Sécurité	SSH keys, UFW, HTTPS/TLS, bastion
Big Data	Ingestion, stockage, analytique, détection anomalies
Monitoring	Netdata centralisé sur bastion

TABLE 9.1 – Couverture des exigences du projet

10. Conclusion

Ce projet nous a permis de concevoir et de déployer une infrastructure virtualisée complète sur une machine personnelle, en appliquant les concepts fondamentaux de la virtualisation : hyperviseur de type 2, machines virtuelles, virtualisation réseau, virtualisation du stockage et conteneurisation.

L'infrastructure met en œuvre une **sécurité multicouche** : bastion host comme unique point d'entrée, authentification par clés SSH, pare-feu UFW avec règles granulaires par IP, chiffrement HTTPS/TLS via Nginx, et contrôle d'accès strict au niveau de PostgreSQL.

Le cas d'usage Big Data démontre que l'infrastructure virtualisée peut supporter un pipeline de données complet : ingestion via API REST, stockage persistant en base de données, traitement analytique avec détection d'anomalies par Z-score, et visualisation en temps réel.

Enfin, la centralisation du monitoring via Netdata sur le bastion offre une visibilité complète sur l'état de l'infrastructure en temps réel, ce qui constitue une pratique professionnelle essentielle dans tout environnement de production.

— *Fin du Rapport* —